MARS Automation Platform

SAPIENZA
UNIVERSITÀ DI ROMA

MSc in Computer Engineering - Laboratory of Advanced Programming 2025/2026

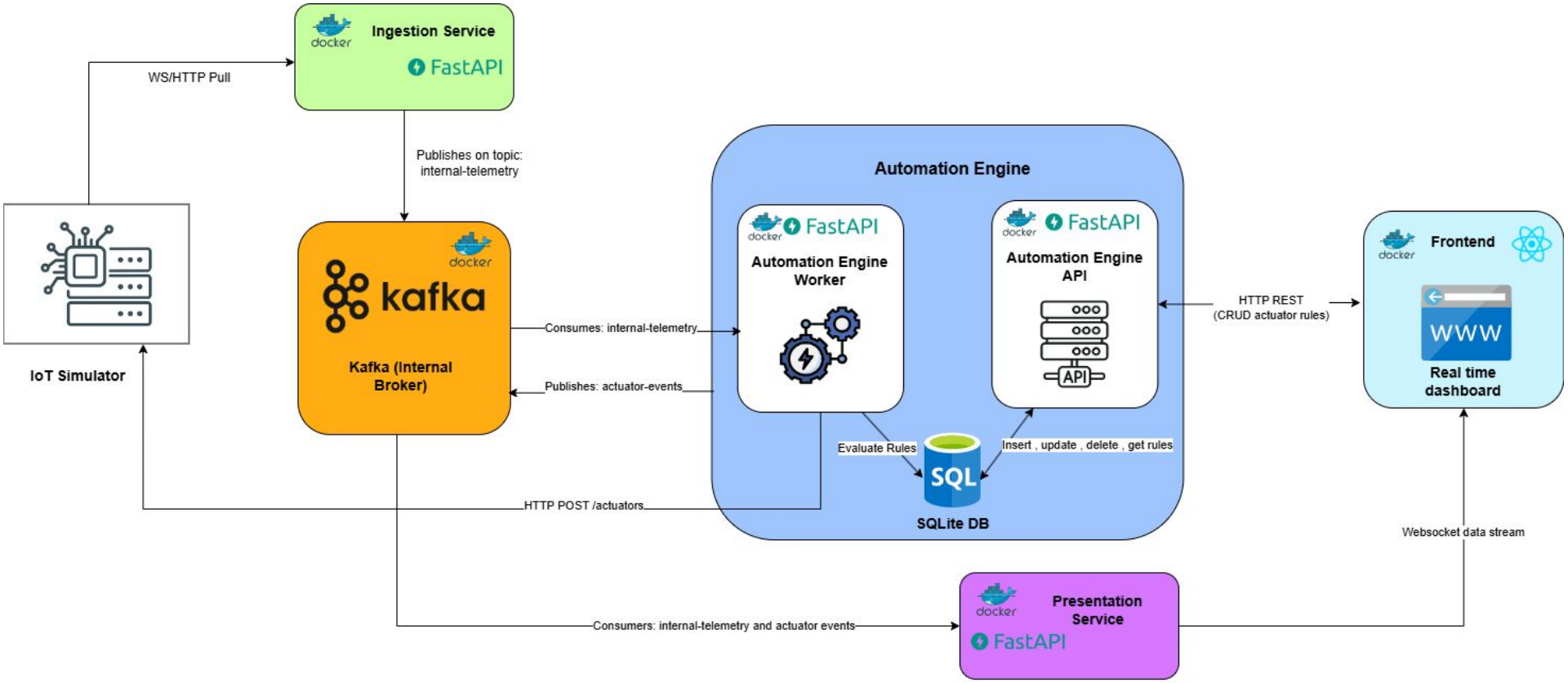
Team: Pasquale Marano [2221228], Roxana Sorina Peto Birlea [1934897], Alberto D'Onofrio [1836590]

The Mission

Rebuild a distributed automation platform capable of:

- ingesting heterogeneous sensor data
- normalizing it into a unified internal representation
- evaluating simple automation rules
- providing a real-time dashboard for habitat monitoring

System Architecture



Ingestion Service: Extraction & Normalization

The bridge between the external IoT Simulator and our internal network.

1 IoT Simulator(REST sensors)

Uses two concurrent tasks to extract data from the IoT Simulator , the first task polls each REST sensor data every 5 seconds

2 IoT Simulator(Stream)

The second task creates a different concurrent task for each telemetry stream and each task connects to the respective websocket stream and listens for data.

3 Data Normalization

For each message received it normalizes it by transforming heterogeneous payloads into a standardized event schema.

4 Data Streaming

After normalizing the payloads it forwards them immediately into the internal-telemetry kafka topic

Apache Kafka Message Broker

Role: Central Message Broker.
It enables asynchronous communication between independent microservices by implementing a publish–subscribe architecture.

Configuration:

- KRaft Mode (Single-node)
- kafka-init helper container that initializes the topics needed to avoid startup race conditions.

Main Benefit of this architecture:

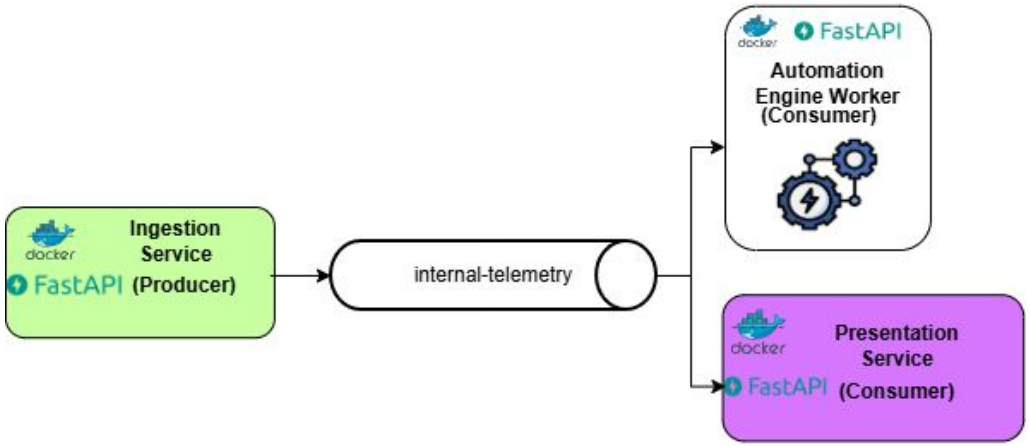
- Complete decoupling of data producers and data consumers.
- Real-time event processing
- Each microservice can be restarted or scaled without breaking the overall system.

Kafka Communication Architecture

Core Topics

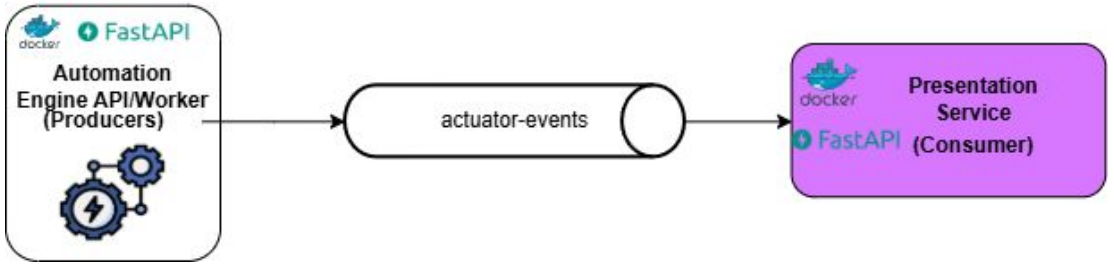
internal-telemetry:

Contains normalized sensor readings produced by the ingestion service.

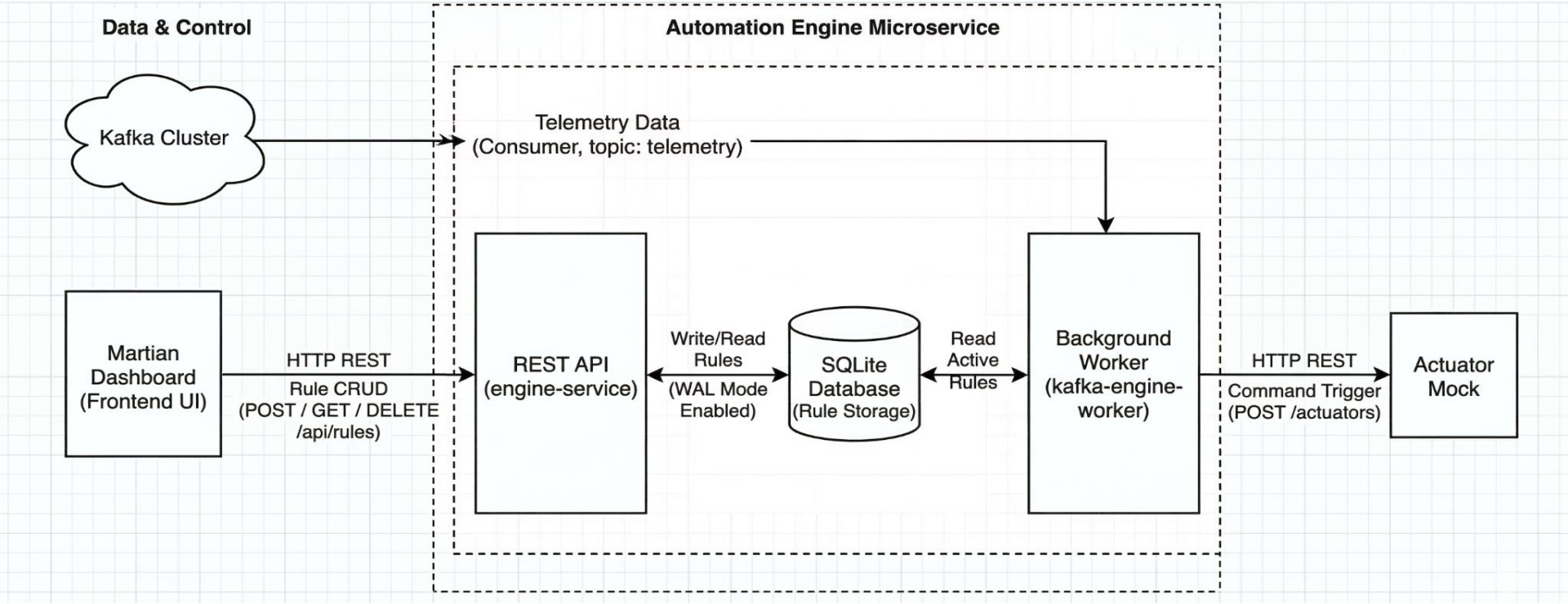


actuator-events:

History of commands sent to the habitat.



Automation Engine: Event-Driven Decision Making



Presentation Service: Real-Time Event Gateway

Design Trade-offs & Details

Database Concurrency

Enabled SQLite WAL (Write-Ahead Logging)

Allows simultaneous API writes and Worker reads without locking the database

Rule Caching Strategy

The worker caches active rules in memory, fetching updates from the DB every 10 seconds

Maximizes telemetry evaluation throughput but introduces a max 10-second propagation delay for new rules

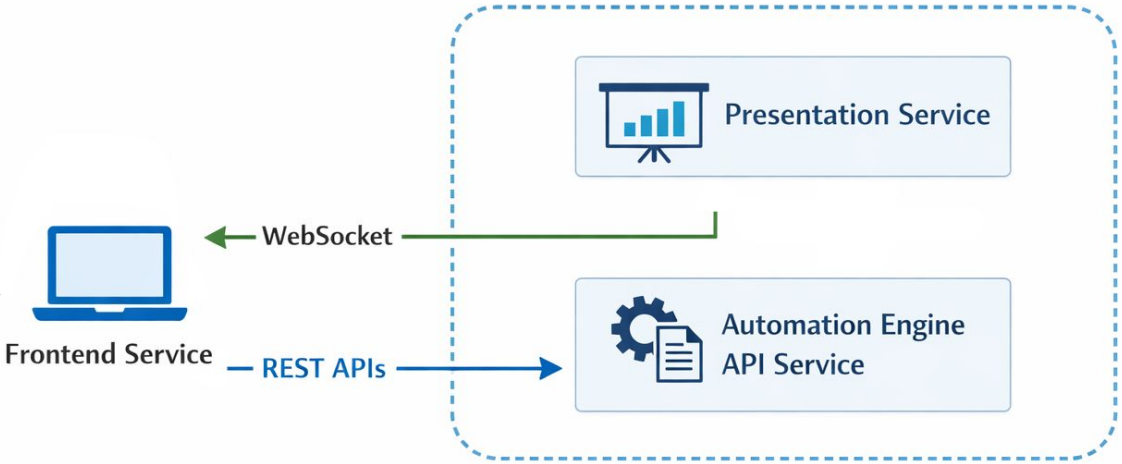
Event Persistence vs. UI Performance

Kafka consumer offset set to latest

Drops pre-connection history to prevent browser flooding upon dashboard reload

Frontend: Service Overview

- React **Single Page Application**
- Web interface for **real-time monitoring**
- Communicates with backend via:
  - **REST API's**: rule management & actuators
  - **WebSocket**: real-time sensor updates
- Maintains temporary **in-memory buffer** for recent telemetry

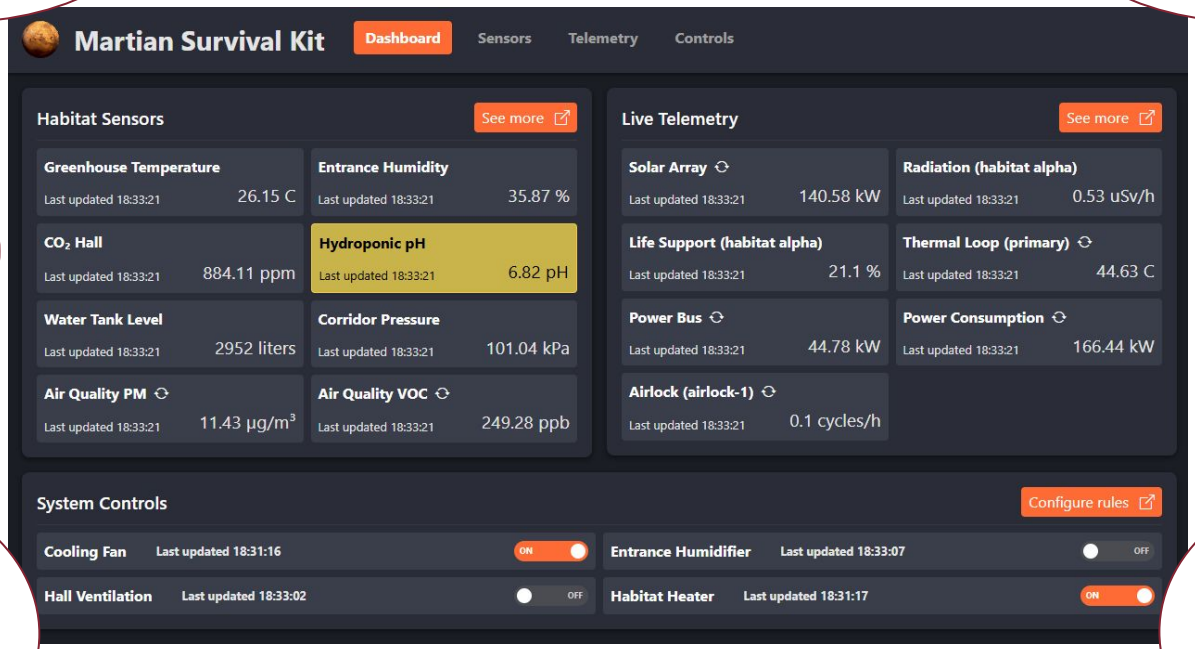


Frontend: Real-Time Dashboard

Overview of the habitat: sensors and actuators

Warning indicators when thresholds exceeded

Updates continuously via WebSocket



Metrics switching

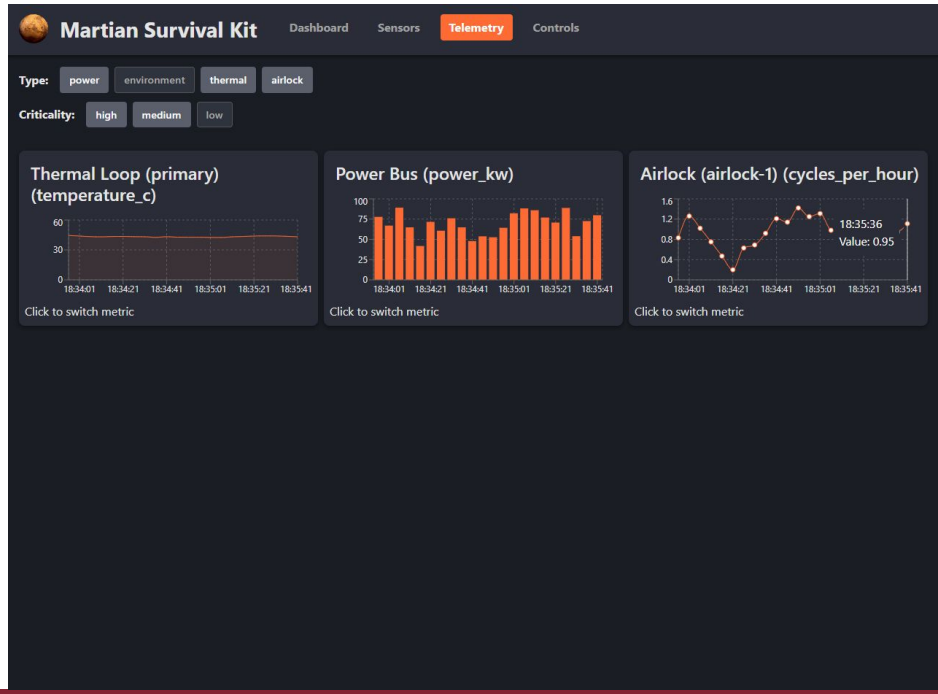
Latest sensor values with timestamp

Actuator states displayed in real-time

Manual actuator toggle from the dashboard

Frontend: Sensor and Telemetry Data Visualization

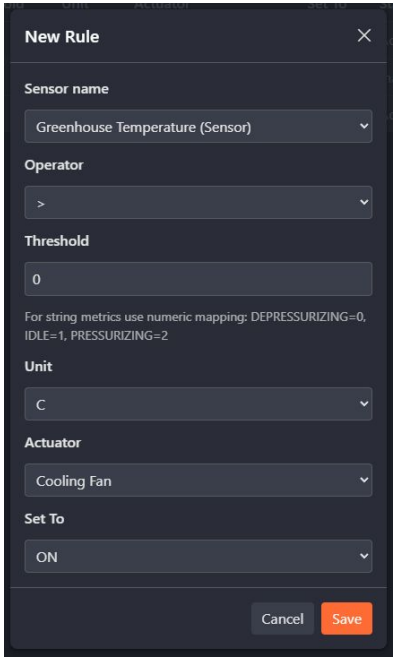
- Dedicated pages for **Sensors** and **Telemetry Topics**
- **Short-term historical visualization** of telemetry data
- Data is displayed through **interactive charts**



- Users can **switch metrics** or measurement units
- **Filters** allow focusing on specific sensors or locations
- Charts **update automatically** via WebSocket events

Frontend: Automation Rules Management

- The Rules page provides a table of all **configured automation rules**
- Users can **create**, **edit**, **delete**, or **disable** rules
- Rule creation and editing are handled through a modal form
- Changes are sent to the Automation Engine via REST APIs



Live Demo

Live demo starting now…